01 / FulfillOps - solo project report

Feras MICHAEL | EPITA DevOps | 11 November 2026 | Repository: github.com/Ferasmich/fulfillops

Project goal

I built an order-processing lab that accepts an order durably, reserves inventory once and completes the order asynchronously. The main problem is correctness during duplicate requests, competing requests, worker crashes and software updates. Solo submission is allowed, as confirmed by the project owner; the required two APIs and two workers are separate processes, not project members.

Architecture

Layer	Implementation
HTTP client -> API Service	Two FastAPI replicas validate requests and accept orders.
Shared PostgreSQL 16	Products, orders, idempotency keys, jobs, attempts and completion receipts are authoritative.
Two workers -> PostgreSQL	Workers claim durable jobs with leases and finish through a protected database transaction.
Git -> Argo CD -> Kubernetes	Approved desired configuration is reconciled into the local kind cluster.
GitHub Actions	Tests, image checks, SBOM and a documented vulnerability policy gate accepted image artifacts.

Python 3.12, Psycopg, pytest/HTTPX and Ruff keep the implementation small. One application image runs either the API or worker command. Docker Compose supports local use; Kubernetes is the deployment lab. There is no frontend, external payment or shipping integration.

Why one database

A PostgreSQL work table avoids a database/message-broker dual-write gap. An accepted order and its job commit together. The tradeoff is polling load and one shared database dependency; persistence is not the same as high availability.

Verification: 74 tests and the full 15-check public checker passed from clean tracked source, with a fresh Python environment and private Compose stack. Evidence paths in this report are relative to the repository.

02 / API, transactions and migrations

Accept an order once

The API validates the idempotency key, JSON, customer and original item quantities. Quantities must be strict integers from 1 to 20; Python booleans are rejected. Duplicate SKU lines are aggregated, then sorted to form a canonical payload. The body is limited to 16 KiB while being read.

One transaction first inserts the unique key mapping. A same-key concurrent insert waits on PostgreSQL uniqueness. If the key already exists, the API compares the saved canonical payload: the same payload returns the original order ID; a changed payload returns 409 IDEMPOTENCY_CONFLICT. The replay branch never reserves stock again.

For a new key, product rows are locked with SELECT FOR UPDATE in SKU order. All requested stock is checked before any decrement. Order, item prices, reservation, key and job commit together. A failure rolls everything back. The API returns 202 only after commit; queued is the acceptance receipt, even when a replay happens after worker completion.

Request/result	Expected behavior
New valid order	202 with stable order_id and queued acceptance receipt
Identical replay	202 with the same order_id, no extra reservation
Changed accepted-key payload	409 IDEMPOTENCY_CONFLICT
Competing request without enough stock	409 OUT_OF_STOCK; no partial order
Unknown order / invalid quantity	404 ORDER_NOT_FOUND / 422 INVALID_QUANTITY

Schema changes

Numbered SQL migrations are applied by the runner, with an advisory lock, recorded version and checksum. Already-applied files are skipped; changed applied files are refused. Normal startup adds only missing test seed products and never restores spent stock. Test reset requires the approved environment, database name, host and confirmation flag.

Evidence: db/migrations/, docs/ADR-001.md, tests/integration/test_api.py and evid/02-api/. Seeds are KIT-100: 7 at 1299 cents; PART-200: 12 at 599 cents; PACK-300: 5 at 249 cents.

03 / Concurrency and worker recovery

Races across two actual API pods

Experiment	Observed result
Eight simultaneous same-key requests	Four sent to each API pod; eight 202 responses, one order ID and one unit reserved.
Twelve different-key requests after that	Five accepted, seven OUT_OF_STOCK, matching the five remaining KIT-100 units in that experiment.
Separate fresh host-process race	Twelve requests competed for seven units: seven accepted, five rejected. These are different initial stocks, not inconsistent results.

The shared transaction decides who gets the last unit. Separate in-memory locks would not coordinate different API replicas. The database CHECK constraint is an additional guard against negative stock, not the whole reservation algorithm.

Durable worker ownership

A worker atomically claims a due or expired job using FOR UPDATE SKIP LOCKED, increments its persisted attempt count and sets a new claim token and lease. It commits the claim before processing. Completion locks the job again, verifies the exact token and an unexpired lease, inserts a unique completion receipt and updates order/job states in one transaction.

Default policy: five attempts, a ten-second lease and exponential backoff. A crashed final attempt is eventually marked failed. Permanently failed orders retain their reserved stock for operator reconciliation; there is no automatic restock or silent deletion.

Controlled crash and poison job

In a separate disposable host test database, a real worker was killed while holding a committed order. A second worker reclaimed it: attempts moved from one to two, retries from zero to one, and exactly one receipt was written. Reserved stock stayed six. API liveness and readiness stayed 200. Shortened test timings are not the default operational lease.

A deterministic poison fixture stopped after three configured attempts and two retries, retaining a failed order and reservation. Exactly-once here means the PostgreSQL completion effect; no external payment or manufacturing action is claimed.

Evidence: evid/07-gitops/cross-pod-races.json; evid/08-reliability/worker-fault/; tests/integration/test_worker.py; tests/races/test_orders.py.

04 / Tests and reproducible containers

Final clean-checkout check

Tracked source snapshot 0f8a7a9 was extracted into a new temporary directory. No working .env, virtual environment or database volume was copied. A new virtual environment installed the pinned development requirements; pip check, Ruff and formatting passed. Random local credentials and separate ports were used for the disposable Compose stack.

Check	Result
Full owned suite	74 passed; one upstream TestClient deprecation warning
Public checker --concurrency --stress	15 passed / 0 failed / 0 skipped on fresh isolated stock
Regression demonstration	A permissive isinstance check accepts True; the test failed, then passed with the strict integer check. Only a temporary source copy was changed.
Compose down/up with volume retained	15 orders, receipts, totals and stock were identical before and after restart.
Cleanup	Only this private final-test project and its volume were removed. The retained Kubernetes database was not reset.

Image and runtime

The multi-stage image installs runtime wheels without retaining the build/cache directory. Development packages and local secrets are excluded. Application containers run as UID/GID 10001 with a read-only filesystem, temporary writable storage, dropped capabilities and no privilege escalation.

Init uses owner credentials for migrations and grants; API and worker use separate restricted roles. The worker cannot reserve stock, and runtime processes cannot perform schema migrations. The earlier negative privilege tests are preserved in Part 5 evidence.

The Part 5 optimized image was 192,614,009 bytes versus 264,756,846 for its naive baseline, about 27.2% smaller. That comparison combines dev-package/cache/root changes; it does not isolate multi-stage building alone or represent download size.

Evidence: evid/09-final/{verification.json,tests.txt,tests.xml,public-checker.txt,compose-persistence.json}; evid/05-containers/. The final local image ID starts sha256:774d06f2796c6912 and is identical to the Part 8 local ARM64 runtime.

05 / CI and supply-chain controls

Quality path

PRs and pushes to main run pinned actions, lint/format checks, owned tests, the deliberate regression exercise, image build, a fresh container checker, a CycloneDX SBOM and Trivy OS/Python scans. Accepted image archives are uploaded only after quality checks pass. Image metadata records source commit and image ID.

Observed CI evidence	Outcome
PR regression run 43071282696	Two tests failed on boolean quantity acceptance. No accepted-image artifact was produced.
PR repair run 43071728762	70 tests and the checker passed after strict validation was restored. PR #1 remains unmerged.
Application candidate run 43084811974	Successful CI for d9a144cca067221ef8f02e3d652548cc85dcf0e4; quality and accepted-image artifacts are listed by GitHub.
Final evidence commit 0f8a7a9	Its workflow was still queued with no jobs at the audit snapshot. It is not reported as a green run.

Blocking versus informational

The policy blocks HIGH/CRITICAL findings that have a vendor-reported fix, malformed/missing output, missing scan coverage and scanner failures. Unfixed High/Critical and lower-severity findings remain visible. The retained baseline CI scan had zero blockers and 351 informational findings, including 96 High and two Critical. These historical numbers are not relabelled as a new scan or a claim of safety.

Trivy runs with the calling user and an owned cache/output directory. The initial scan failure was a Linux archive-read permission error, not proof that a vulnerability gate rejected the image. The scanner has no Docker socket, repository credentials or .env mount.

Artifact publication scope

The accepted candidate is a GitHub build artifact named accepted-image-d9a144cca067221ef8f02e3d652548cc85dcf0e4-1. Registry promotion is a separate manual workflow, disabled by default and dependent on quality. No GHCR push or registry digest is claimed. The setup guide explicitly permits local kind image loading; the deployed ARM64 rebuild is distinguished from the AMD64 CI binary.

Evidence: .github/workflows/ci.yml; docs/security-policy.md; evid/06-ci/github/; evid/09-final/application-ci-artifacts.json. Downloading the newer artifact returned HTTP 502 during this audit; earlier downloaded CI SBOM/scan files remain preserved.

06 / Kubernetes and GitOps

Deployment

The owned kind-fulfillops cluster runs two API replicas, two independent worker pods and PostgreSQL with a PVC. Manifests include Service, ConfigMap, Secret references, probes, requests/limits and restricted security contexts. Argo CD reads main from the real public repository and manages only the configured namespace/resources.

Versions used: kind v0.33.0, Kubernetes v1.37.0 and Argo CD v3.6.1. The Argo installer and base images are pinned. Local Secret values are generated outside Git and preserved with database storage. The public repository needs no source token.

Desired state and observed health

Synced means managed live configuration matches Git. Healthy means resource health checks are satisfied. A configuration can be Synced and still be Degraded, as the bad readiness experiment showed. Argo checks software/deployment configuration, not whether a physical customer car has the right options.

The Part 7 drift drill temporarily paused self-healing, scaled the API from two to three and captured OutOfSync. Restoring the committed self-healing configuration made Argo return to two and Synced/Healthy. There was no final manual Deployment patch. Automatic pruning is disabled and the PVC has an extra prune guard.

Persistence

Deleting the PostgreSQL pod changed its pod UID, while the same PVC and underlying volume were retained. All 21 completed orders and 21 receipts, saved totals and stock survived. Replaying the proof key returned the same acceptance receipt. This proves pod replacement persistence, not recovery after deleting the entire cluster.

Safe rollout strategy

The API uses maxUnavailable 0, maxSurge 1, minReadySeconds 3 and a 90-second progression deadline. Readiness controls Service endpoints; liveness stays independent. A three-second preStop delay allows endpoint removal to propagate before shutdown.

Evidence: infra/k8s/, infra/argocd/, docs/kubernetes.md and evid/07-gitops/. The setup guide was corrected to load the currently referenced d9a144c image. Compose on 8080 and the Kubernetes port-forward on 8081 use separate databases; spent Kubernetes test stock is expected to stay zero.

07 / Observability and incident recovery

Measured local window

The API exposes HTTP volume, created/failed orders, worker retries and a latency histogram. Request logs contain timestamp, method, route template, status and duration; customer bodies, headers, keys and query strings are omitted. Business counters are shared database totals: use max across API pods, not sum. Per-process HTTP counters can be summed.

Experiment / target	Observation
250 sequential GET /api/v1/catalog requests	250 successes, zero failures; nearest-rank p95 = 54.54 ms (sorted observation 238).
Proposed local-window targets	At least 99.5% success and p95 below 300 ms; both met in this short test.
Good release: supplied 30-second probe	120 successes / zero failures; probe overlapped real API pod replacement.
Bad-release 120-second probe	476 successes / zero failures; covers the fault and start of revert, ending before final recovery gate.

API incident timeline (UTC)

Time	Action / observation
20:18:35 -> 20:19:08	Good release pushed, then healthy acceptance confirmed.
20:19:23	Bad readiness URL pushed: /health/forced-not-ready returns 404.
20:21:12 -> 20:21:15	ProgressDeadlineExceeded; Argo Degraded captured. Bad pod excluded from ready Service endpoints; both healthy old pods available.
20:21:17 -> 20:21:36	Git revert pushed; expected revision, two updated/ready replicas and Synced/Healthy verified. All 21 orders/receipts preserved.

Rollback was appropriate because only configuration changed and the previous revision was known good; no schema/data rollback was required. Kubernetes did not edit Git automatically. The explicit deployment gate rejected acceptance, and the drill performed the Git revert. Image CI alone did not detect the wrong manifest URL.

Prevention: keep readiness gating and old-pod availability; add a manifest probe-URL smoke check. This local sequential read test does not prove monthly SLOs, POST latency or production load. Client latency includes the tunnel and no-retry proxy. Evidence: evid/08-reliability/ and docs/incident-postmortem.md.

08 / Limits, responsibility and submission

What the lab does not prove

One node and one PostgreSQL instance do not provide high availability, backups or disaster recovery. There is no user authentication/order ownership check, production TLS/secret manager or centralized monitoring service. UUIDs do not replace authorization. Metrics access is appropriate only inside the isolated lab. Long-running worker tasks would need conditional lease renewal.

Exactly-once completion is scoped to one database receipt. External payment, shipping or manufacturing would require an idempotent downstream operation and an appropriate integration design. Unfixed scan findings remain a security risk. Registry publication has not been exercised.

Solo responsibility and assistance

I am presenting the complete system alone and am responsible for understanding its architecture, code paths, concurrency, failure recovery and operational choices. My recorded manual work includes Python/database setup and API/worker checks. I directed the staged project work. Codex assisted with implementation, tests, automation, verification and documentation; the actual scope is recorded in docs/assistance-log.md. The instructor documents and checker remain supplied course materials. This does not attribute the whole project or its direction to AI.

Presentation and final actions

The assignment permits a demo video or live presentation of at most 15 minutes; the setup checklist suggests 12-15 minutes. The solo plan targets 13 minutes 30 seconds, including short demonstrations, with 90 seconds of margin. Required proof includes a failing test, replay, both races, crash/poison handling, rolling Git recovery and the incident timeline. Individual defense can include a random code path or unfamiliar broken pod.

The implementation and local evidence checks are complete. The final documentation workflow remains queued at this audit snapshot; confirm its actual outcome before claiming a fully green submission. A final checker screenshot, presentation rehearsal and actual instructor submission are user-facing handoff actions, not silently assumed completed.

References and evidence map

Course: assessment/FulfillOps_Assignment_EN.md sections 3, 5 and 7-9; SETUP_AND_SUBMISSION_EN.md sections 8-9; API_CONTRACT_EN.md. Original copies are for reference, not content to include in a student-only ZIP.

Implementation references: PostgreSQL 16 SELECT/INSERT documentation; Psycopg 3 transaction contexts; FastAPI tutorial; Docker multi-stage/Compose startup guidance; Trivy SBOM documentation. Links are in docs/ADR-001.md, docs/part5-notes.md and docs/security-policy.md. Project evidence index: evid/INDEX.md. Full postmortem: docs/incident-postmortem.md. Fresh final checker and JUnit: evid/09-final/.